p0919R1
Heterogeneous lookup for unordered containers

Published Proposal, 1 April 2018

This version:
http://wg21.link/P0919r1

Author:
Mateusz Pusz (Epam Systems) mateusz.pusz@gmail.com

Audience:
LWG

Project:
ISO JTC1/SC22/WG21: Programming Language C++

Source:
github.com/mpusz/wg21_papers/blob/master/src/unordered_heterogeneous_lookup.bs

---

Abstract

This proposal adds heterogeneous lookup support to the unordered associative containers in the C++ Standard Library. As a result, a creation of a temporary key object is not needed when different (but compatible) type is provided as a key to the member function. This also makes unordered and regular associative container interfaces and functionality more compatible with each other.

With the changes proposed by this paper the following code will work without any additional performance hits:

```
template<typename Key, typename Value>
using h_str_umap = std::unordered_map<Key, Value, string_hash>;
h_str_umap<std::string, int> map = /* ... */;
map.find("This does not create a temporary std::string object :-)"sv);
```

Table of Contents

§ 1. Motivation and Scope

[N3657] merged into C++14 IS introduced heterogeneous lookup support for ordered associative containers (`std::map`, `std::set`, etc) to the C++ Standard Library. Authors of that document pointed that the requirement to construct (either implicitly or explicitly) the object of `key_type` to do the lookup may be really expensive.

Unordered containers still lack support for such functionality and users are often hit by that performance problem.

§ 1.1. Performance related concerns

Consider such use case:

EXAMPLE 1

```
std::unordered_map<std::string, int> map = /* ... */;
auto it1 = map.find("abc");
auto it2 = map.find("def"sv);
```

In C++17 above code will construct `std::string` temporary and then will compare it with container's elements to find the key. There is no implementation-specific reason to prevent lookup by an arbitrary key type `T`, as long as `hash(t) == hash(k)` for any key `k` in the map, if `t == k`.

§ 1.2. Design related concerns

Another motivating case is mentioned in [N3573]. Consider:

EXAMPLE 2

```
std::unordered_set<std::unique_ptr<T>> set;
```

Whilst it's possible to insert `std::unique_ptr<T>` into the set, there are no means to erase or test for membership, as that would involve constructing two `std::unique_ptr<T>` to the same resource.

In such a case C++ developer is forced to either:

1. Weaken the design and not use smart pointers for memory ownership management which may result int stability or security issues.
2. Provide custom stateful (memory overhead) deleter that only optionally destroys the managed resource as suggested by [STACKOVERFLOW-1]:

```
class opt_out_deleter {
  bool delete_;
public:
  explicit opt_out_deleter(bool do_delete = true) : delete_{do_delete} {}
  template<typename T>
  void operator()(T* p) const
  {
    if(delete_) delete p;
  }
};

template<typename T>
using set_unique_ptr = std::unique_ptr<T, opt_out_deleter>;

template<typename T>
set_unique_ptr<T> make_find_ptr(T* raw)
{
  return set_unique_ptr<T>{raw, opt_out_deleter{false}};
}

set_unique_ptr set = /* ... */;
auto it = set.find(make_find_ptr(raw));
```

3. Use `std::unordered_map<T*, std::unique_ptr<T>>` instead which again results in memory overhead.

The similar code may also have a different side effect. Let’s consider:

EXAMPLE 3

```
struct my_data {
  size_t i;
  std::array<char, 256> data;
  explicit my_data(size_t i_) : i{i_}
  { std::iota(begin(data), end(data), 0); }
};

struct my_data_equal {
  bool operator()(const std::unique_ptr<my_data>& l,
                  const std::unique_ptr<my_data>& r) const
  { return l->i == r->i; }
};

struct my_data_hash {
  size_t operator()(const std::unique_ptr<my_data>& v) const
  { return std::hash<size_t>{}(v->i); }
};

using my_set = std::unordered_set<std::unique_ptr<my_data>,
                                  my_data_hash, my_data_equal>;
my_set set = /* ... */;
auto it = set.find(std::make_unique<my_data>(1));
```

This case not only introduces a dynamic memory allocation related performance hit on every lookup but also messes up with nicely defined ownership strategy.

§ 2. Prior Work

[N3573] tried to address this issue. While the motivation described in that paper sounds reasonable the proposed solution goes too far and may cause problems. See §4 Design Decisions for more details.

§ 3. Impact On The Standard

This proposal modifies the unordered associative containers in `<unordered_map>` and `<unordered_set>` by overloading the lookup member functions with member function templates.

There are no language changes.

Almost all existing C++17 code is unaffected because new member functions are disabled from overload resolution process unless `Hash` template parameter has `transparent_key_equal` property. That is not the case for the code created before this proposal.

§ 4. Design Decisions

§ 4.1. Heterogeneous hash function object

[N3573] paper suggests adding

```
namespace std {
  template<typename T = void>
  struct hash;

  template<>
  struct hash<void> {
    template<typename T>
    std::size_t operator()(T&& t) {
      return std::hash<typename std::decay<T>::type>()(std::forward<T>(t));
    }
  };
}
```

While that could be useful and compatible with changes introduced for many operations in [N3421], there is too big chance of two types being equality-comparable but having incompatible hashes.

Following issue was pointed out in the [REFLECTOR-1].

For example, under gcc 7.2.0,

```
std::hash<long>{}(-1L) == 18446744073709551615ULL
std::hash<double>{}(-1.0) == 11078049357879903929ULL
```

which makes following code fail

```
std::unordered_set<double, std::hash<>, std::equal_to<>> s;
s.insert(-1L);                  // Internally converts -1L to -1.0
assert(s.find(-1L) != s.end()); // Fails: calls hash<long>(-1L) and gets the wrong bucket
```

Note that under C++17 rules that code succeeds, because `find()` also converts its parameter to `double` before hashing.

This proposal intentionally **does not suggest** standardizing heterogeneous hash function object `template<> std::hash<void>`. Doing that might be tempting but it requires more investigations and can be always added via future proposals.

§ 4.2. Additional parameters in lookup member functions overloads

[N3573] also proposes adding additional parameters to lookup functions so the users may provide different hash and equality comparison functor objects for each member function call.

```
template<typename T, typename Hash = std::hash<>, typename Eq = std::equal_to<>>
iterator find(T t, Hash h = Hash(), Eq e = Eq());
template<typename T, typename Hash = std::hash<>, typename Eq = std::equal_to<>>
const_iterator find(T t, Hash h = Hash(), Eq e = Eq()) const;
```

That is not consistent with the current interface of ordered associative containers and therefore it is **not proposed** by this paper. If such functionality is considered useful it can be added in the future by other paper both for ordered and unordered associative containers.

§ 4.3. Lookup member functions template overloads

For consistency reasons this paper proposes heterogeneous lookup for unordered associative containers should be provided by the similar means as it is the case for ordered ones. Containers **will only change their interface when hash functor will define nested tag type called** `transparent_key_equal` that specifies transparent equality comparator type to be used by the container **instead of a type provided (or default type) for** `Pred` **template parameter**.

The container **will fail to compile** (with proper diagnostics applied) when:

- equality comparator type provided by the `hasher::transparent_key_equal` is not transparent (does not provide `is_transparent` tag type)
- `Pred` container's template argument is neither
  - the default type of that template parameter (namely `equal_to<Key>`)
  - the same type as provided by the hasher via `transparent_key_equal` tag

`key_equal` **member type of the container** will specify either:

- the type provided by the `Pred` template argument of the container (or its default type) in case the heterogeneous lookup is disabled
- the type provided by the `transparent_key_equal` tag of the hash functor object otherwise

> Note: Changing the specification of the default type in container's template parameters would cause the ABI break, therefore, it is not suggested by that proposal.

By providing explicit tag `transparent_key_equal` in the hash functor object, the user explicitly states that the intention of this type is to provide coherent and interchangeable hash values for all the types supported by the functor's call operators.

Concerns raised in §1 Motivation and Scope are addressed by this proposal in the following way:

```
struct string_hash {
  using transparent_key_equal = std::equal_to<>;  // Pred to use
  using hash_type = std::hash<std::string_view>;  // just a helper local type
  size_t operator()(std::string_view txt) const   { return hash_type{}(txt); }
  size_t operator()(const std::string& txt) const { return hash_type{}(txt); }
  size_t operator()(const char* txt) const        { return hash_type{}(txt); }
};

std::unordered_map<std::string, int, string_hash> map = /* ... */;
map.find("abc");
map.find("def"sv);
```

Note that in the above example the 4th template argument (`Pred`) is intentionally skipped and will be overwritten with the type provided by the `string_hash::transparent_key_equal`.

In case the user needs to provide custom `Allocator` type the `Pred` arguments needs to match the type provided by `hasher::transparent_key_equal`:

```
std::unordered_map<std::string, int, string_hash,
                   string_hash::transparent_key_equal,
                   std::allocator<std::pair<const std::string, int>>> map = /* ... */;
```

To find more details on how to address all code examples provided in this paper please refer to §7 Implementation Experience.

§ 5. Proposed Wording

The proposed changes are relative to the working draft of the standard as of [n4700].

Modify **26.2.7 [unord.req]** paragraph 11 as follows:

> In Table 91: `X` denotes an unordered associative container class, `a` denotes a value of type `X`, `a2` denotes a value of a type with nodes compatible with type `X` (Table 89), `b` denotes a possibly `const` value of type `X`, `a_uniq` denotes a value of type `X` when `X` supports unique keys, `a_eq` denotes a value of type `X` when `X` supports equivalent keys, `a_tran` denotes a possibly `const` value of type `X` when the *qualified-id* `X::hasher::transparent_key_equal` is valid and denote a type (17.9.2), `i` and `j` denote input iterators that refer to `value_type`, `[i, j)` denotes a valid range, `p` and `q2` denote valid constant iterators to `a`, `q` and `q1` denote valid dereferenceable constant iterators to `a`, `r` denotes a valid dereferenceable iterator to `a`, `[q1, q2)` denotes a valid range in `a`, `il` denotes a value of type `initializer_list<value_type>`, `t` denotes a value of type `X::value_type`, `k` denotes a value of type `key_type`, `ke` is a value such that (1) `eq(r, ke) == eq(ke, r)` with `r` the key value of `e` and `e` in `a_tran`, (2) `hf(r) == hf(ke)` if `eq(r, ke)` is `true`, and (3) `eq(r, ke) && eq(r, r') == eq(r', ke)` where `r'` is also the key of an element in `a_tran`, `hf` denotes a possibly `const` value of type `hasher`, `eq` denotes a possibly `const` value of type `key_equal`, `n` denotes a value of type `size_type`, `z` denotes a value of type `float`, and `nh` denotes a non-const rvalue of type `X::node_type`.

Modify table 91 in section **26.2.7 [unord.req]** as follows:

Expression	Return type	Assertion/note pre-/post-condition	Complexity
X::key_equal	~~Pred~~ `Hash::transparent_key_equal` if the *qualified-id* `Hash::transparent_key_equal` is valid and denotes a type (17.9.2); otherwise, `Pred`.	Requires: `key_equal` is `CopyConstructible`. `key_equal` shall be a binary predicate that takes two arguments of type `Key`. `key_equal` is an equivalence relation. `key_equal::is_transparent` is valid and denotes a type (17.9.2) if `Hash::transparent_key_equal` is valid and denotes a type (17.9.2)	compile time
...			
`b.find(k)`	`iterator`; `const_iterator` for const `b`.	Returns an iterator pointing to an element with key equivalent to `k`, or `b.end()` if no such element exists.	Average case O(1), w case O(`b.size()`).
`a_tran.find(ke)`	`iterator`; `const_iterator` for const `a_tran`.	Returns an iterator pointing to an element with key `r` such that `eq(r, ke)`, or `a_tran.end()` if no such element exists.	Average case O(1), w case O(`a_tran.size()`)
`b.count(k)`	`size_type`	Returns the number of elements with key equivalent to `k`.	Average case O(`b.count(k)`), w case O(`b.size()`).
`a_tran.count(ke)`	`size_type`	Returns the number of elements with key `r` such that `eq(r, ke)`.	Average case O(`a_tran.count(` worst case O(`a_tran.size()`)
`b.equal_range(k)`	`pair<iterator, iterator>`; `pair<const_iterator, const_iterator>` for const `b`.	Returns a range containing all elements with keys equivalent to `k`. Returns `make_pair(b.end(), b.end())` if no such elements exist.	Average case O(`b.count(k)`), w case O(`b.size()`).
`a_tran.equal_range(k)`	`pair<iterator, iterator>`; `pair<const_iterator, const_iterator>` for const `a_tran`.	Returns a range containing all elements with keys `k` such that `eq(k, ke)` is `true`. Returns `make_pair(a_tran.end(), a_tran.end())` if no such elements exist.	Average case O(`a_tran.count(` worst case O(`a_tran.size()`)

Add new paragraphs (18, 19) in **26.2.7 [unord.req]**:

> The program is ill-formed and requires proper diagnostic if the *qualified-id* `Hash::transparent_key_equal` is valid and denotes a type (17.9.2) and either *qualified-id* `Hash::transparent_key_equal::is_transparent` is not valid or `Pred` is a different type than `equal_to<Key>` or `Hash::transparent_key_equal`.

> The member function templates `find`, `count`, and `equal_range` shall not participate in overload resolution unless the *qualified-id* `Hash::transparent_key_equal` is valid and denotes a type (17.9.2).

Modify **26.5.4.1 [unord.map.overview]** paragraph 3 as follows:

```
namespace std {
  template<class Key,
           class T,
           class Hash = hash<Key>,
           class Pred = equal_to<Key>,
           class Allocator = allocator<pair<const Key, T>>>
  class unordered_map {
  public:
    // types
    using key_type         = Key;
    using mapped_type      = T;
    using value_type       = pair<const Key, T>;
    using hasher           = Hash;
    using key_equal        = Pred;
    using key_equal        = see Table 91;
    using allocator_type   = Allocator;
```

```
// map operations:
iterator       find(const key_type& k);
const_iterator find(const key_type& k) const;
template <class K> iterator       find(const K& k);
template <class K> const_iterator find(const K& k) const;
size_type count(const key_type& k) const;
template <class K> size_type count(const K& k) const;
pair<iterator, iterator>             equal_range(const key_type& k);
pair<const_iterator, const_iterator> equal_range(const key_type& k) const;
template <class K> pair<iterator, iterator>             equal_range(const K& k);
template <class K> pair<const_iterator, const_iterator> equal_range(const K& k) const;
```

In **26.5.5.1 [unord.multimap.overview]** add:

```
namespace std {
  template<class Key,
           class T,
           class Hash = hash<Key>,
           class Pred = equal_to<Key>,
           class Allocator = allocator<pair<const Key, T>>>
  class unordered_multimap {
  public:
    // types
    using key_type         = Key;
    using mapped_type      = T;
    using value_type       = pair<const Key, T>;
    using hasher           = Hash;
    using key_equal        = Pred;
    using key_equal        = see Table 91;
    using allocator_type   = Allocator;
```

```
// map operations:
iterator       find(const key_type& k);
const_iterator find(const key_type& k) const;
template <class K> iterator       find(const K& k);
template <class K> const_iterator find(const K& k) const;
size_type count(const key_type& k) const;
template <class K> size_type count(const K& k) const;
pair<iterator, iterator>             equal_range(const key_type& k);
pair<const_iterator, const_iterator> equal_range(const key_type& k) const;
template <class K> pair<iterator, iterator>             equal_range(const K& k);
template <class K> pair<const_iterator, const_iterator> equal_range(const K& k) const;
```

In **26.5.6.1 [unord.set.overview]** add:

```
namespace std {
  template<class Key,
           class Hash = hash<Key>,
           class Pred = equal_to<Key>,
           class Allocator = allocator<pair<const Key, T>>>
  class unordered_set {
  public:
    // types
    using key_type         = Key;
    using value_type       = Key;
    using hasher           = Hash;
    ~~using key_equal        = Pred;~~
    using key_equal        = see Table 91;
    using allocator_type   = Allocator;
```

```
// set operations:
iterator       find(const key_type& k);
const_iterator find(const key_type& k) const;
template <class K> iterator       find(const K& k);
template <class K> const_iterator find(const K& k) const;
size_type count(const key_type& k) const;
template <class K> size_type count(const K& k) const;
pair<iterator, iterator>             equal_range(const key_type& k);
pair<const_iterator, const_iterator> equal_range(const key_type& k) const;
template <class K> pair<iterator, iterator>             equal_range(const K& k);
template <class K> pair<const_iterator, const_iterator> equal_range(const K& k) const;
```

In **26.5.7.1 [unord.multiset.overview]** add:

```
namespace std {
  template<class Key,
           class Hash = hash<Key>,
           class Pred = equal_to<Key>,
           class Allocator = allocator<pair<const Key, T>>>
  class unordered_multiset {
  public:
    // types
    using key_type         = Key;
    using value_type       = Key;
    using hasher           = Hash;
    using key_equal        = Pred;
    using key_equal        = see Table 91;
    using allocator_type   = Allocator;
```

```
// set operations:
iterator       find(const key_type& k);
const_iterator find(const key_type& k) const;
template <class K> iterator       find(const K& k);
template <class K> const_iterator find(const K& k) const;
size_type count(const key_type& k) const;
template <class K> size_type count(const K& k) const;
pair<iterator, iterator>             equal_range(const key_type& k);
pair<const_iterator, const_iterator> equal_range(const key_type& k) const;
template <class K> pair<iterator, iterator>             equal_range(const K& k);
template <class K> pair<const_iterator, const_iterator> equal_range(const K& k) const;
```

§ 6. Feature Testing

The `__cpp_lib_unordered_map_heterogeneous_lookup` feature test macro should be added.

§ 7. Implementation Experience

Changes related to this proposal as well as answers to all of the code examples provided in this paper are partially implemented in GitHub repo against libc++ 5.0.0.

Simple performance tests provided there proved more than:

- 20% performance gain for short text (SSO used in `std::string` temporary) in EXAMPLE 1
- 35% performance gain for long text (dynamic memory allocation in `std::string` temporary) in EXAMPLE 1
- 85% performance gain in EXAMPLE 3

§ 8. Possible Future Extensions

§4.1 Heterogeneous hash function object and §4.2 Additional parameters in lookup member functions overloads are not proposed by this paper but can be explored and proposed in the future.

§ 9. Revision History

§ 9.1. r0 ➡ r1 [diff]

The paper was reviewed by LEWG at the 2018 Jacksonville meeting and resulted with the following straw polls

- Do we want encourage to do more work in that subject? Unanimous consent.
- Put the comparator in the hash (and forbid specifying it as an explicit 4th template parameter).

SF	F	N	A	SA
6	8	2	0	2

r1 changes the way the transparent equality comparator is provided to the class template. Instead of depending on the user to do the right thing in providing both hasher and comparator that are transitive and compatible with each other, now the design forces hasher to provide compatible comparator.

§ 10. Acknowledgements

Thanks to Casey Carter for initial review of this proposal and help with wording issues.

Special thanks and recognition goes to Epam Systems for supporting my membership in the ISO C++ Committee and the production of this proposal.

§ References

§ Normative References

[N3657]
J. Wakely, S. Lavavej, J. Muñoz. Adding heterogeneous comparison lookup to associative containers (rev 4). 19 March 2013. URL: https://wg21.link/n3657

[N4700]
Richard Smith. Working Draft, Standard for Programming Language C++ Note:. URL: https://wg21.link/n4700

§ Informative References

[N3421]
Stephan T. Lavavej. Making Operator Functors greater<>. 20 September 2012. URL: https://wg21.link/n3421

[N3573]
Mark Boyall. Heterogenous extensions to unordered containers. 10 March 2013. URL: https://wg21.link/n3573

[REFLECTOR-1]
Joe Gottman. N3573: Why heterogenous extensions to unordered containers will not work. URL: https://groups.google.com/a/isocpp.org/d/msg/std-proposals/mRu7rIrDAEw/bYMyojZRaiEJ

[STACKOVERFLOW-1]
Xeo. Using a std::unordered_set of std::unique_ptr. URL: https://stackoverflow.com/a/17853770